

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Developing a secure cloud-based system that manages patients' information require knowledge from various areas, which involve threats faced by healthcare information systems, architecture principles used in cloud computing, patients' data regulations, and DevSecOps processes that ensure that the system is developed and deployed in a secure manner. This chapter reviews the knowledge on how the proposed system can be designed are reviewed. This is done through first considering the case study organization before moving to general technological and literature.

Section 2.2 introduces the case study organization, Alamin Clinic, through analysing its organizational structure, current manual operating model, and the interview outcomes from key people in clinic. The section also analyzes the difference between the current followed process and the process followed when the new system is introduced in the clinic. Section 2.3 carries out a 5W 1H analysis on the problems faced by the current system. Section 2.4 compares the proposed system to existing healthcare systems, including the unique aspects from each existing healthcare management system. Section 2.5 highlights the major technologies used in designing the new system and their supporting literature. Section 2.6 presents key conclusions from the literature review.

2.2 Case Study: Alamin Clinic (Saudi Arabia)

Alamin Clinic is a private independent health care service in Saudi Arabia. In essence, this is the organization to be considered a stakeholder and used as the main case study for this proposal. The incident that has happened to Alamin Clinic and led to the development of this idea was an attack by ransomware on their IT services.

The patient base of Alamin Clinic includes people who seek various types of consultations, follow-up sessions, and health record administration services. The key stakeholders, which include doctors, administrative staff, and patients themselves, all have different ways of interacting with the database of patients in the clinic, and therefore, will need different access control mechanisms. This diversity of roles makes the design of a Role-Based Access Control (RBAC) model a fundamental requirement of the proposed system.

2.2.1 Alamin Clinic: Organizational Structure

There is a three-level organization hierarchy in Alamin Clinic that is common among small private clinics in the region. At the clinical level, there are general specialist physicians, which include doctors that will consult the patient, diagnose the illnesses, and manage the patient's medical records. The physician will need read and write access to the medical record and will not have access to the administrative record like billing and salary of employees.

At the administrative level, there are people that will register and schedule patients. Administrators should have access to registration and scheduling only and should not be allowed to see medical records or any other information.

It is important to ensure separation of duties as this will keep confidential the patients' data and will be made possible through the recommended IAM framework. Patients that are registered in Alamin Clinic can login into the system to read their medical records and appointments.

2.2.2 Manual Operation

The existing system used at Alamin Clinic for storing patient information follows a traditional on-premises design model defined by the following characteristics: manual server configuration, manual deployment, local hosting, and reactive security.

Manual Server Configuration. The server hardware installed at the clinic is configured locally by IT staff of the clinic. Server configuration is done manually without maintaining infrastructure versions and automating server deployment

processes. This methodology is Unreliable because it relies on manual configuration of servers with no control over drift. It is impossible to roll back the configuration in case of failure because no automated server provisioning is used (Paidy and Chaganti, 2024).

Manual Deployment. The code deployed in the development environment is manually moved to the production server with the use of either FTP clients or storage devices, such as USB drives. This methodology slowdown the deployment time and prevents implementing automatic checks of the code for security or quality issues before deploying it to the production environment. Furthermore, the manual deployment process ensures that malware can enter the production environment bypassing any controls.

Local Hosting. The frontend of the website, the backend logic of the application, and the patient database are all hosted on the same physical servers. There is no network segmentation between these layers, which means that gaining access to any of the components is enough to access the rest. The absence of network segmentation and isolation is considered one of the key factors that contribute to ransomware infections among healthcare facilities, allowing the malicious code to move directly to the database layer (Argaw *et al.*, 2019; Thamer and Alubady, 2021).

Reactive Security. Firewalls and antivirus solutions are updated manually by IT staff according to available information about existing threats. This practice does not provide enough protection against emerging cyberattacks because updates are carried out reactively and are therefore lagging behind the threat landscape. As a result, known vulnerabilities are patched late, exposing the facility to attacks.

In effect, it was shown by the fact that the clinic's security was compromised after a ransomware attack, where the hackers managed to encrypt all the patients' information, making the records inaccessible for use by the clinic. As such, it was evident that there were no protective measures to block an intruder from reaching their target data.

2.2.3 Interview with Clinic Stakeholders

In order to get first hand information about the problems faced by the clinic and weaknesses of the system, two major stakeholders of the clinic were interviewed in person through a video conferencing. These include one Administrative Staff member who is involved in registering patients and scheduling appointments and a Clinical Staff member who performs routine operations at the clinic. Considering the privacy concerns of the participants, there was no audio recording of the interviews; instead, notes were made while conducting the interviews. There were a total of eight open-ended questions in the interview protocol.

The full interview instrument is provided in Appendix C. Selected findings are summarised below in table 2.1.

Table 2.1 Summary of Stakeholder Interview Findings

Question Theme	Administrative Staff	Clinical Staff
Daily Access	Individual logins for a 3rd-party desktop app via external vendor. Limited remote access.	Same desktop app; managed by provider. No in-house IT capability for diagnosis.
Main Difficulties	No unique patient IDs cause duplication. Slow billing. Manual Excel tracking for appointments.	Inconsistent entry compounding duplication. No automated monitoring or alerting for the server.
Ransomware Discovery	All data inaccessible. Operations halted. Used paper forms as fallback.	All departments frozen simultaneously. Doctors could not access files; appointments cancelled.
Downtime Duration	72 hours of total blackout. Restoration failed due to corrupted or incomplete data.	5-day recovery window. Significant backlog accumulated before operations resumed.
Data Lost	All patient records inaccessible. No clean backup existed to restore the system.	Records in the attack period were permanently lost. Access unavailable during recovery.
Security Measures	Basic antivirus with manual updates. No formal backup policy or scheduled reviews.	No formal security policy or staff awareness of recovery procedures.

Question Theme	Administrative Staff	Clinical Staff
Improvements	Web interface, auto-notifications, remote access, unique IDs, and a public website.	Dept-specific modules, automated workflows, and faster performance during consultations.
Migration Concerns	Data sovereignty and internet reliability in the clinic's operating region.	Data privacy, access control clarity, and training requirements for the new system.

2.2.4 Current System Workflow

The current process for managing patient information in Alamin Clinic relies entirely on manual handling without any automated means. Figure 2.2 highlights the flowchart for managing the main processes for patient information from the arrival of patients to storing their records.

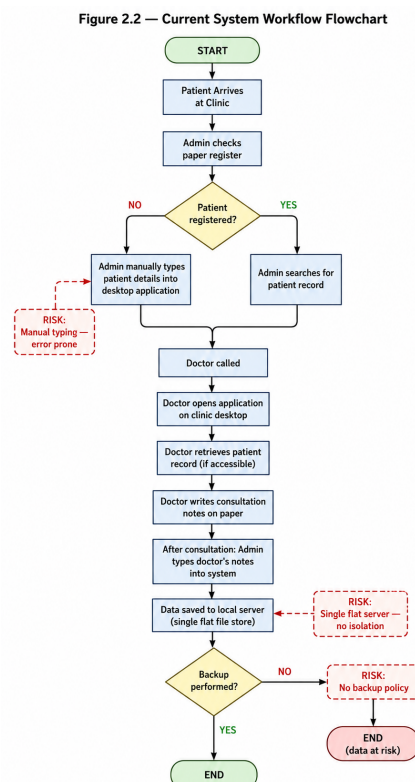


Figure 2.1 Current System Workflow Flowchart

There are five significant threats associated with the current workflow process. Firstly, all inputs into the process are made manually by administrative staff, resulting in errors and time wastage. Secondly, there is no distinction between the application that processes the information and the database that holds the data because both use the same server process and file structure. Thirdly, there is no distinction between the logins for physicians and administrative staff since both log into the same application with the security measures depending purely on trust. Fourthly, there is no automatic backup system for the process, meaning that all information storage depends entirely on the availability of the actual server. Lastly, there is no way to track who accessed or changed what in patients' files.

2.2.5 Proposed System Workflow

The proposed system replaces each manual, insecure step in the current workflow with an automated, security-enforced equivalent. Figure 2.3 presents the flowchart of the proposed system's patient record management process.

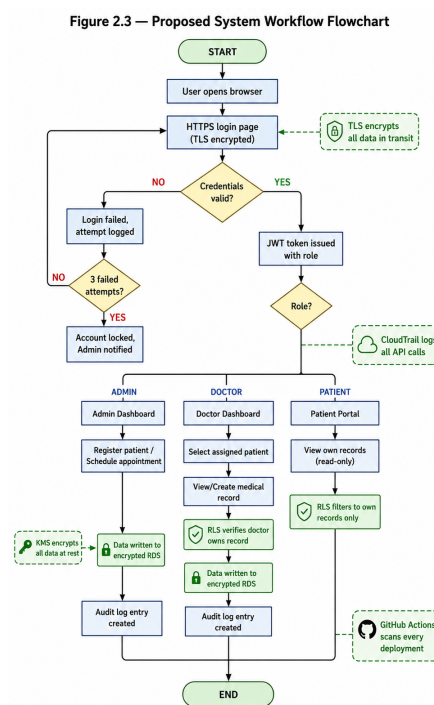


Figure 2.2 Proposed System Workflow Flowchart

The solution presented here solves all of the mentioned vulnerabilities in the system. Instead of manual data entry, there is now an input-validation web page. In addition, the single-server design is swapped for a three-tier VPC architecture in which the database has been separated into a private subnet that does not have internet access. Instead of using role enforcement, the authentication mechanism relies on JWT and PostgreSQL row-level security to control who accesses what data. Physical hardware backup is no longer used; instead, the system will automatically create snapshots in RDS and redeploy the entire infrastructure with Terraform. Finally, audit logging has been introduced via CloudTrail and an audit table in the application.

Table 2.2 presents a side-by-side comparison of the current and proposed workflows across the five identified vulnerability dimensions.

Table 2.2 Comparison Between the Current System and the Proposed Cloud-Based System

Dimension	Current System	Proposed System
Data entry	Manual typing by admin, error-prone	Structured web forms with validation
Architecture	Single flat server, all tiers co-hosted	Three-tier VPC — presentation, app, DB isolated
Authentication	Single shared login, trust-based	JWT tokens + RBAC enforced at app & DB layers
Backup and recovery	No backup policy; full data loss risk	Automated RDS snapshots + Terraform redeploy in < 15 min
Audit trail	None	CloudTrail API logs + application audit_log table
Deployment	Manual FTP/USB, no security gate	GitHub Actions CI/CD with Trivy, SonarQube, & Checkov
Encryption	None at rest; inconsistent in transit	KMS AES-256 at rest; TLS 1.2+ in transit

2.3 Current System Analysis

2.3.1 Problem Analysis Using the 5W 1H Framework

The 5W 1H framework — Who, What, When, Where, Why, and How — provides a structured approach that ensures all dimensions of the issue are addressed before a solution is designed. Table 2.3 applies this framework to the Alamin Clinic problem.

Table 2.3 5W1H Analysis of the Current System Vulnerabilities

Dimension	Analysis
Who is affected?	Doctors cannot access patient records or create diagnoses. Administrative staff cannot manage appointments. Patients cannot receive care during outages. The clinic director faces operational and potential legal liability for data loss.
What is the problem?	A ransomware attack encrypted the patient database. No recovery capability or backups existed. The systemic problem is an infrastructure designed without security controls at any layer.
When does it exist?	At all times. The server is exposed to the internet with no isolation. Patches are reactive, and every manual deployment is a security risk.
Where does it originate?	In the flat network architecture of the on-premise server, providing no barrier to lateral movement. In the absence of network segmentation between web, app, and database layers.
Why does it persist?	Small healthcare providers lack the budget for enterprise security platforms and under-resourced IT staffing. This is the market gap the proposed system addresses.
How will it be solved?	Through a security-by-design cloud architecture: (1) Three-tier VPC isolation. (2) Terraform IaC for configuration consistency. (3) DevSecOps pipeline (Trivy, SonarQube, & Checkov). (4) IAM/RBAC with PostgreSQL security. (5) KMS encryption & TLS in transit. (6) AWS Security Hub for HIPAA compliance.

2.3.2 Security Domain Analysis

The assessment of the existing infrastructure of the Alamin Clinic, before Starting on the migration process, shows that there are weaknesses in the four areas of security:

Network Architecture. In the case of the clinic, the network architecture is highly simplified in that all three tiers of applications—the web, application, and database are hosted in one machine with one network interface, No DMZ for internet connected parts of the network, and no way to prevent lateral movements once the system is breached. This completely violates the concept of defense in depth because it does not offer more than one line of defense (Al-Issa *et al.*, 2019).

Access Control. The user access to the system is determined based on the authentication carried out only at the application level without having any kind of identity management. There are role separations between doctors, administration, and patients that are implemented in the application level but do not have any restrictions at the networking or storage level.

Data Security. The patient data stored in the database of the clinic is not encrypted in its static form. The transmission of the data from the browser of the user to the server is not reliably encrypted using TLS due to the lack of analysis of the HTTP settings used by the clinic. Both the static and the dynamic data are exposed to the threat of being intercepted and stolen(Shojaei *et al.*, 2024).

Operational Resilience. There is no official data backup policy at the clinic, nor is there any officially documented way to recover from such situations. This scenario has been noted in many studies involving ransomware attacks on other small health care providers.Argaw *et al.* (2019) noted that the lack of backup and recovery policy contributed to prolonged disruption during such incidents.

In combination, all of the flaws above shows that the existing model does not meet even the basic criteria of security for a healthcare information system.

2.4 Comparison between existing systems

To frame the proposed model among other solutions for managing healthcare, a comparative table 2.4 is presented below that shows how the proposed solution fits alongside three other systems, namely, a conventional HMS system, the OpenEMR system, the Epic Systems solution, and the proposed Secure Cloud PDMS.

Table 2.4 Comparison of Existing Healthcare Management Systems

System	Key Features	Security Model	Main Limitations
Traditional On-Premise	Full data control, no internet dependency.	Reactive, manual patches, perimeter firewall.	High ransomware risk, no isolation, poor scalability.
OpenEMR	Patient portal, billing, EHR.	Reactive, community patches.	Manual security updates, no auto-deployment.
Epic Systems	Comprehensive clinical suite, HL7/FHIR.	Proactive, vendor-managed updates, IAM.	Prohibitive cost for small clinics, vendor lock-in.
Proposed System	Patient records, roles, cloud-native.	Proactive, shift-left, automated scanning, RBAC.	Infrastructure focus; billing/pharmacy out of scope.

The comparison highlights a unique weakness of the existing market towards small healthcare providers. On-premise software applications and open-source projects like OpenEMR provide high levels of flexibility but require all aspects of security management to be undertaken by the IT staff of the medical facility, as proven by the history of the Alamin Clinic. However, even though commercial products, including those by Epic Systems, provide an advanced level of security, they are too expensive and complicated for small private providers.

This project will fill this niche through a combination of a low-cost model associated with the use of cloud hosting and the application of automation technologies used in commercial enterprise-level systems. The inclusion of the infrastructure as code through Terraform and the automatic scanning of security issues during the CI/CD process will reduce the need for manual IT procedures, thus minimizing this primary weakness of the existing solution.

2.4.1 Features Adopted from Existing Systems

The comparison provided in Table 2.4 made it possible to choose the features from the existing systems, which would be relevant to the goals of the suggested system both in terms of scope and security considerations. The features borrowed from the existing systems and the modifications made are shown in Table 2.5.

Table 2.5 Features Adopted from Existing Systems

Feature	Adopted From	Justification
Patient Registration	OpenEMR	Core requirement; directly addresses ransomware data loss.
Appointment Scheduling	OpenEMR	Operational necessity; appointment data was highly disruptive when lost.
Patient Portal	OpenEMR	Reduces admin load; addresses patient autonomy.
RBAC Model	Epic Systems	Enterprise-grade pattern validated by Singh & Chatterjee (2015).
Audit Logging	Epic Systems	HIPAA requirement; key missing finding from incident review.
HIPAA Framework	Epic Systems	Recognized benchmark for healthcare data security.

Another difference of the system under discussion from its competitors lies in the comparative analysis of the standard AWS architecture design template. Table 2.6 compares these two types of templates.

Table 2.6 Proposed System vs Standard AWS Architecture

Feature	Standard AWS Architecture	Proposed Solution
Deployment	Manual or basic scripts	Fully automated via Terraform IaC
Security Model	Added after build (Reactive)	Shift-left scanned at every CI/CD stage
Post-attack Recovery	Long manual rebuild process	Instant redeployment from Terraform state
Data Protection	Basic private subnets	Hardened NACLs + Private RDS + KMS encryption
Compliance	Ad hoc manual checks	HIPAA posture measured via AWS Security Hub

2.5 Literature Review of Technology Used

2.5.1 Cloud Computing in Healthcare

Since the early 2010s, the use of cloud computing technology within healthcare has been extensively researched. The study performed by Ahuja *et al.* (2012), which can be considered one of the first comprehensive investigations of cloud computing usage in healthcare settings, concludes that cost savings, scalability, and improved data access are the most common factors contributing to widespread adoption, whereas data protection, regulatory issues, and vendor reliability stand out as the key barriers. All those findings still hold true today. In particular, the security problems mentioned by Ahuja *et al.* (2012), such as concerns about data sovereignty and access control, have influenced the design of the IAM policy for our solution.

In the same spirit, Al-Issa *et al.* (2019) investigated cloud security issues specific to eHealth environments. Their survey revealed that unauthorized access, data corruption, and service availability issues are the leading threats affecting eHealth clouds. Accordingly, each threat mentioned in their study is addressed by the corresponding design requirement in our case: unauthorized access is controlled via

IAM/RBAC; integrity violations are prevented using CloudTrail audit logging and KMS encryption; and availability is guaranteed with a multi-AZ architecture and fast recovery provided by Terraform.

The stakeholder concerns regarding cloud data privacy raised by the clinic stakeholders in section 2.2.3 interviews are handled by the proposed solution with the use of (a) the AWS region chosen to be in gulf region according to PDPL data residency rules, (b) IAM least privilege principles, and (c) row-level security of PostgreSQL database which ensures that one cannot access another user's data regardless of infrastructure level access(Ramayanam, 2025).

2.5.2 Three-Tier Architecture

The three-tier architectural approach involves breaking down a web application into three different layers which are physically and logically segregated from one another, namely the presentation tier (frontend), application tier, and data tier (database). There are two key advantages to using this model from a security perspective. First, the damage scope for a breach in a particular tier is significantly reduced. Second, it allows for the independent scaling and access control to be implemented at the level of each individual tier.

In this particular case, the three-tier architecture is deployed using the AWS Virtual Private Cloud (VPC). In accordance with the model, the presentation tier uses an Application Load Balancer running on a public subnet while the application tier uses EC2 instances running on the private application subnet. Finally, the database tier utilizes Amazon RDS, residing in the dedicated private database subnet. The fact that this tier cannot be accessed directly from the Internet can be seen as a built-in protection against the network flatness exploited during the attack at Alamin Clinic.

The proposed design differs from the current design in that it makes use of a three-tiered structure compared to a single-server flat approach used now (Section 2.2.4). The new design includes three tiers that will each have their own separate security group policy, as well as having the database without an outbound Internet route(Gaber *et al.*, 2025).

2.5.3 AWS and the Shared Responsibility Model

The Amazon Web Services runs on a shared responsibility model where there are clearly defined boundaries about who is accountable for what part of the security. It states:

“AWS manages security of the cloud, you are responsible for security in the cloud.”

This means that AWS will take care of the physical security of data centers, the hypervisor level, and the managed services itself. While the customer—the clinic in the case is accountable for OS management, Network Security Groups rules, IAM policies, application security, and data encryption.

It is crucial in the design process for the new system since moving away from traditional hosting to the cloud doesn't relieve the clinic of any security concerns; instead, it shifts them around. It changes the list of those tasks that have been historically difficult for Alamin Clinic to manage manually to the set that can be easily automated using Terraform, KMS, and IAM policies(Mendoza and Reyes, 2023).

2.5.4 Infrastructure as Code (Terraform)

IaC refers to the use of machine-readable configuration files in lieu of manual procedures to describe and provision computing infrastructure. According to Paidy and Chaganti (2024), IaC significantly decreases configuration drift, making the infrastructure failure recoverability fast and reliable in multi-region AWS deployments.

The chosen IaC tool is HashiCorp's Terraform, which supports a declarative configuration approach that allows the representation of the whole topology of the VPC network, EC2 instance settings, RDS parameter groups, IAM policies, and security group rules in version-controlled code. In effect, the entire environment could be rebuilt from scratch in minutes that allows turning the Recovery Time Objective into a quantifiable metric.

The discovery directly correlates with the most important insight gained during the stakeholder interview when the IT Administrator was asked about how long it takes to recover from the ransomware attack, and the answer was a period of five days. The recommended RTO target of the system should be less than fifteen minutes, which is possible only due to the configuration being scripted in Terraform and deployable to a clean AWS environment with one command.

Checkov is a static code analyzer tool for IaC that is included in the CI/CD pipeline to scan Terraform scripts before the infrastructure is put to production to avoid common infrastructures security pitfalls, such as excessive permissions on security groups or unprotected storage volumes(Verdet *et al.*, 2023; Espinha Gasiba *et al.*, 2021).

2.5.5 DevSecOps and Shift-Left Security

DevSecOps refers to the process of incorporating security testing and verification during the software development and deployment process, as opposed to mandating security as a gate for the software development and deployment process after development(Valdés-Rodríguez *et al.*, 2023). “Shift-left” involves moving security tests from their current position closer to the development stage, where issues are detected and fixed at the least cost, even before they become part of the production code base (Paidy and Chaganti, 2024).

In the current deployment process, there is no security gate at any stage. This process involves pushing the code to the production server using FTP. In the new system, there is a CI/CD pipeline in GitHub Actions that carries out security scans in three types on each commit:

1. **Trivy:** Scans Docker container images for known Common Vulnerabilities and Exposures (CVEs) before any image is pushed to the container registry.
2. **SonarQube:** Performs Static Application Security Testing (SAST) by analyzing the application source code for security anti-patterns, injection vulnerabilities, and code quality issues.

3. **Checkov:** Analyzes Terraform configuration files for infrastructure-level security misconfigurations, ensuring that the defined AWS resources adhere to best practices

Any pipeline breach during a critical discovery ensures that the software will not be released, guaranteeing that no software containing a critical flaw is ever deployed into the production environment. The move from no security gate to an automated blocking gate in three stages will solve the reactive security stance described in Section 2.2.2(Rajapakse *et al.*, 2022; Akbar *et al.*, 2022).

2.5.6 Identity and Access Management and Role-Based Access Control

IAM refers to the policies, procedures, and technologies that control the access to certain resources by a particular user within the system. IAM on AWS provides granular and rule-based control over access to infrastructure components, thereby complementing the application-based RBAC mechanism.

Singh and Chatterjee (2015) proved that for effective security management in the cloud computing environment, there is a need for security enforcement across various tiers, ranging from the application login page to network and resource levels. This paper's proposed framework follows this approach, as it combines AWS IAM policies, which restrict the API calls available to each application module, with application-level RBAC, which restricts the data that each authorized user can read/write, and PostgreSQL Row Level Security, which enforces isolation of data at the storage level, irrespective of the application level.

Three IAM roles will be created for the intended system according to the clinic's three user groups – Doctor, Admin, and Patient. Specifically, the Doctor role will have read/write access to their patients' information, the Admin role will be allowed to read/write information concerning scheduling and registrations, and the Patient role will only have read access to their own information. Each IAM role will have minimum privileges sufficient for their intended purpose(Butt *et al.*, 2023).

PostgreSQL row-level security as an additional control measure will address the issue raised in Section 2.3 regarding inadequate controls on the access aspect of the current system design since the application-based role access controls offer zero security against direct access to the underlying database, which is what ransomware attackers exploit(Cobrado *et al.*, 2024).

2.5.7 HIPAA Compliance

The Health Insurance Portability and Accountability Act (HIPAA) is the main regulatory framework used in securing and protecting patient health information within the United States and is considered a gold-standard benchmark for healthcare data security across the world. HIPAA Security Rule outlines the technical requirements of access control, audit logging, integrity of data, and encryption during data transmission, all of which relate directly to the IAM, CloudTrail, KMS, and TLS of the proposed system(Abbasi and Smith, 2024).

AWS Security Hub is an AWS-managed service designed to provide continuous posture assessment of the AWS environment in relation to HIPAA compliance and provides detailed assessments where there are discrepancies between the configuration and the standards. The suggested architecture uses AWS Security Hub to measure and assess HIPAA posture compliance.

The current framework does not meet the criteria for any of the technical safeguards prescribed by HIPAA, which includes failure to provide proper access control at the infrastructure level, failure to maintain an audit log of data access activities, absence of data integrity through encryption, and lack of reliable transmission security measures. The recommended framework caters to all four areas, thus positioning HIPAA readiness as the benchmark for the assessment process.

2.6 Chapter Summary

This chapter provides the theoretical basis and comparison of the proposed secure cloud-based system for managing patient data. Four major vulnerabilities of the current legacy on-premises solution were identified based on structured interviews conducted as part of the Alamin Clinic case study: the flat structure of the network,

poor access control, lack of data encryption, and weak infrastructure resilience. All those factors combined helped the ransomware breach and led to five days of downtime and considerable data loss. The 5W1H technique was applied to categorize all the aforementioned problems along six axes.

To address these problems, workflow analysis outlines the transformation from the current manual solution to a new, secure architecture. Main improvements include moving from a flat network to three-tier VPC, switching from FTP deployments to the DevSecOps CI/CD pipeline, implementation of zero trust access control using JWT and Row-Level Security (RLS) in PostgreSQL, reduction of infrastructure recovery time to 15 minutes using Terraform, and implementing continuous audit with AWS CloudTrail. The thorough literature review proves these solutions according to academic and industry standards and ensures HIPPA-compliant technology stack using AWS Security Hub. Finally, the chapter explains the methodology used to develop the project.